

Lyra

René Ferdinand Rivera Morell – grafikrobot@gmail.com – 1.6.1

Table of Contents

1. Introduction
 - 1.1. License
 - 1.2. Features
2. Usage
 - 2.1. Single Option
 - 2.2. Multiple Options
3. Alternate Specification
4. Help Option
5. Value Choices
6. Option Values
7. Argument Groups
8. Sub-commands
9. Main
10. More Main
11. Counting Flags
12. Option Styles
13. Tail Arguments
14. Reference
 - 14.1. `lyra::arg`
 - 14.2. `lyra::args`
 - 14.3. `lyra::arguments`
 - 14.4. `lyra::cli`
 - 14.5. `lyra::command`
 - 14.6. `lyra::exe_name`
 - 14.7. `lyra::group`
 - 14.8. `lyra::help`
 - 14.9. `lyra::literal`
 - 14.10. `lyra::main`
 - 14.11. `lyra::opt`
 - 14.12. `lyra::option_style`
 - 14.13. `lyra::parser_result`
 - 14.14. `lyra::parser`
 - 14.15. `lyra::composable_parser`
 - 14.16. `lyra::bound_parser`
 - 14.17. `lyra::printer`
 - 14.18. `lyra::ostream_printer`
 - 14.19. `lyra::val`
15. History
 - 15.1. 1.8

15.2. 1.7
15.3. 1.6.1
15.4. 1.6
15.5. 1.5.1
15.6. 1.5
15.7. 1.4
15.8. 1.3
15.9. 1.2
15.10. 1.1
15.11. 1.0

1. Introduction

Lyra is a simple to use, composing, command line parser for C++ 11 and beyond. It provides easy to use command line parsing for most use cases with a minimal source footprint. It doesn't aim to provide all features for all users.

1.1. License

Distributed under the highly permissive Boost Software License, Version 1.0. (See accompanying file LICENSE.txt or copy at http://www.boost.org/LICENSE_1_0.txt)

1.2. Features

- Header only with no external dependencies (except the `std` library).
- Define your interface once to get parsing, type conversions and usage strings with no redundancy.
- Composing. Each `opt` or `arg` is an independent parser. Combine these to produce a composite parser—this can be done in stages across multiple function calls—or even projects.
- Bind parsers directly to variables that will receive the results of the parse—no intermediate dictionaries to worry about.
- Or can also bind parsers to lambdas for more custom handling.
- Deduces types from bound variables or lambdas and performs type conversions (via `ostream <<`), with error handling, behind the scenes.
- Bind parsers to vectors for args that can have multiple values.
- Uses result types for error propagation, rather than exceptions (doesn't yet build with exceptions disabled, but that will be coming later)
- Models POSIX standards for short and long opt behavior.
- Customizable option syntax.
- Specify cardinality of `arg`-s from one to many.
- Limit option values to a specified set of values.

2. Usage

To use, just `#include <lyra/lyra.hpp>`

2.1. Single Option

A parser for a single option can be created like this:

```
#include <cstdlib>
#include <iostream>
#include <lyra/lyra.hpp>

int main(int argc, const char** argv)
{
    // Where we read in the argument value:
    int width = 0;

    // The parser with the one option argument:
    auto cli = lyra::cli()
        | lyra::opt(width, "width")
        ["-w"]["--width"]("How wide should it be?");

    // ...
```

C++

You can use this parser by giving it the program arguments of `main`:

```
// ...

// Parse the program arguments:
auto result = cli.parse({ argc, argv });

// Check that the arguments were valid:
if (!result)
{
    std::cerr << "Error in command line: " << result.message() << std::endl;
    return 1;
}

std::cout << "width = " << width << "\n";
return 0;
}
```

C++

Which could be used as:

```
> example1 -w 10
width = 10
```

SHELL

2.2. Multiple Options

It's rare that we are interested in accepting a single option. To parse multiple options we compose the options as alternatives with the `or` operator (`|`).

C++

```
#include <cstdlib>
#include <iostream>
#include <lyra/lyra.hpp>

int main(int argc, const char** argv)
{
    // Where we read in the argument values:
    int width = 0;
    std::string name;
    bool doIt = false;

    // The parser with the multiple option arguments. They are composed
    // together by the "|" operator.
    auto cli
        = lyra::opt(width, "width")
          ["-w"]["--width"]("How wide should it be?")
        | lyra::opt(name, "name")
          ["-n"]["--name"]("By what name should I be known")
        | lyra::opt(doIt)
          ["-d"]["--doit"]("Do the thing");

    // ...
}
```

You can use this parser by giving it the program arguments of `(main)`:

```
// ...

// Parse the program arguments:
auto result = cli.parse({ argc, argv });

// Check that the arguments where valid:
if (!result)
{
    std::cerr << "Error in command line: " << result.message() << std::endl;
    return 1;
}

std::cout << "width = " << width << ", name = " << name << ", doIt = " << doIt << "\n";
return 0;
}
```

C++

Which could be used as:

```
> example2 -w 10 --name=Lyra
width = 10, name = Lyra, doIt = 0
```

SHELL

3. Alternate Specification

Using operators for specifying the parser argument is not everyone's "cup of tea". Like all such interface choices there are advantages and disadvantages to any particular API one chooses. Because of this Lyra supports both the preceding `(operator)` model and a "named arguments using methods" model for specifying the arguments. Below are the various specification operations and corresponding operator and method call equivalents:

Help description	<code>parser("Help")</code>	<code>parser.help("Help")</code>
Optional argument	<code>parser.optional()</code>	<code>parser.optional()</code>
Required argument	<code>parser.required()</code>	<code>parser.required()</code>
Range of arguments	<code>parser.cardinality(n,m)</code>	<code>parser.cardinality(n,m)</code>
Value choices	<code>parser.choices(a,b,c)</code>	<code>parser.choices(a,b,c)</code>
Add argument	<code>parser = argument</code>	<code>parser.add_argument(argument)</code>
Add option name	<code>option["--name"]</code>	<code>option.name("--name")</code>
Help option description	<code>help.description("Description")</code>	<code>help.description("Description")</code>

The method names try to follow the well known Python `argparse` nomenclature for familiarity. Using the alternative methods one could rewrite the previous two examples. The first one with the single option would be:

```
#include <cstdlib>
#include <iostream>
#include <lyra/lyra.hpp>

int main(int argc, const char** argv)
{
    // Where we read in the argument value:
    int width = 0;

    // The parser with the one option argument:
    auto cli = lyra::cli();
    cli.add_argument(
        lyra::opt(width, "width")
            .name("-w")
            .name("--width")
            .help("How wide should it be?"));

    // Parse the program arguments:
    auto result = cli.parse({ argc, argv });

    // Check that the arguments where valid:
    if (!result)
    {
        std::cerr << "Error in command line: " << result.message() << std::endl;
        return 1;
    }

    std::cout << "width = " << width << "\n";
    return 0;
}
```

C++

And the second with multiple options would the become:

C++

```
#include <cstdlib>
#include <iostream>
#include <lyra/lyra.hpp>

int main(int argc, const char** argv)
{
    // Where we read in the argument values:
    int width = 0;
    std::string name;
    bool doIt = false;

    // The parser with the multiple option arguments. They are composed
    // together by the "|" operator.
    auto cli = lyra::cli();
    cli.add_argument(lyra::opt(width, "width")
        .name("-w").name("--width").help("How wide should it be?"));
    cli.add_argument(lyra::opt(name, "name")
        .name("-n").name("--name").help("By what name should I be known"));
    cli.add_argument(lyra::opt(doIt)
        .name("-d").name("--doit").help("Do the thing"));

    // Parse the program arguments:
    auto result = cli.parse({ argc, argv });

    // Check that the arguments were valid:
    if (!result)
    {
        std::cerr << "Error in command line: " << result.message() << std::endl;
        return 1;
    }

    std::cout << "width = " << width << ", name = " << name << ", doIt = " << doIt << "\n";
    return 0;
}
```

4. Help Option

From the specified arguments parser we also get convenient help output.

```

int main(int argc, const char** argv)
{
    // Where we read in the argument values:
    int width = 0;
    std::string name;
    bool doIt = false;
    bool show_help = false;

    // The parser with the multiple option arguments and help option.
    auto cli
        = lyra::help(show_help) 2
        | lyra::opt(width, "width")
            ["-w"]["--width"]("How wide should it be?")
        | lyra::opt(name, "name")
            ["-n"]["--name"]("By what name should I be known")
        | lyra::opt(doIt)
            ["-d"]["--doit"]("Do the thing");

    // ...

```

- 1 Flag variable to indicate if we get the `(-h)` or `(--help)` option.
- 2 The help specific option parser.

We need some changes when using the parser to check if the help option was specified:

```

// ...

// Parse the program arguments:
auto result = cli.parse({ argc, argv });

// Check that the arguments where valid:
if (!result)
{
    std::cerr << "Error in command line: " << result.message() << std::endl;
    std::cerr << cli << "\n"; 1
    return 1;
}

// Show the help when asked for.
if (show_help) 2
{
    std::cout << cli << "\n";
    return 0;
}

std::cout << "width = " << width << ", name = " << name << ", doIt = " << doIt << "\n";
return 0;
}

```

- 1 We print out the help text on error.
- 2 And we also print it out when specified.

5. Value Choices

For value arguments, i.e. `--name=value` or positional arguments, one can specify a set of allowed values. You can indicate the choices explicitly by specifying them to the `choices()` call:

```
#include <iostream>
#include <lyra/lyra.hpp>

int main(int argc, const char** argv)
{
    std::string choice;
    // Ex: <exe> --choice=red
    auto cli = lyra::cli()
        | lyra::opt(choice, "-c")["--choice"]
          .choices("red", "green", "blue");
    auto result = cli.parse({ argc, argv });
    if (result)
    {
        std::cout << "Your preferred color is " << choice << "\n";
        return 0;
    }
    else
    {
        std::cerr << result.message() << "\n";
        return 1;
    }
}
```

C++

Or you can specify a complex set of values by giving `choices()` a function that validates if a given value is allowed:

```
#include <iostream>
#include <lyra/lyra.hpp>

int main(int argc, const char** argv)
{
    int choice = 5;
    // Ex: <exe> --choice=3
    auto cli = lyra::cli()
        | lyra::opt(choice, "-c")["--choice"]
          .choices([](int value) { return 1 <= value && value <= 10; });
    auto result = cli.parse({ argc, argv });
    if (result)
    {
        std::cout << "Your number between one and ten is " << choice << "\n";
        return 0;
    }
    else
    {
        std::cerr << result.message() << "\n";
        return 1;
    }
}
```

C++

For either case the default, initial, value is only modified if a valid value is given.

6. Option Values

For options that are bound to values, and hence that accept a user value on the command line, various styles of option+value syntax is allowed for the user as follows:

Long options	<code>opt(v, "value").name("--option")</code>	<code>--option VALUE</code> , <code>--option=VALUE</code>
Short options	<code>opt(v, "value").name("-o")</code>	<code>-o VALUE</code> , <code>--o=VALUE</code> , <code>-oVALUE</code>

7. Argument Groups

There are times when some CLI arguments only make sense when considered as a collection. A common case for this are commands. But it also comes up in any case where some arguments are all required, but only if one of them is present. Lets take the example of specifying the dimension of a window. For example one could expect both `--width=X` and `--height=Y` options. But you could make them optional, as a group, in favor of an automatic screen sized window of a particular aspect ratio with a `--aspect=R` option. In such a case we would want to allow either one of these to be specified:

```
exec --width=1440 --height=720
exec --aspect=1.78
```

C++

With argument groups we can allow such an arrangement.

C++

```
#include <lyra/lyra.hpp>

#include <iostream>
#include <string>
#include <vector>

int main(int argc, const char ** argv)
{
    // Default to showing a full screen 4/3 aspect
    bool show_full_screen = true;
    float aspect = 4.0f / 3.0f;

    // If it's not full screen the size will be specified.
    unsigned width = 0;
    unsigned height = 0;

    // Did the user ask for help?
    bool get_help = false;

    lyra::cli cli;
    cli.add_argument(lyra::help(get_help))
        .add_argument(lyra::opt(aspect, "aspect") 1
            .name("--aspect")
            .help("Full-screen aspect ratio window.))
        .add_argument(lyra::group([&](const lyra::group &) {
            show_full_screen = false;
        }) 2
            .add_argument(lyra::opt(width, "width") 3
                .required()
                .name("--width")
                .help("Width of window.))
            .add_argument(lyra::opt(height, "height")
                .required() 4
                .name("--height")
                .help("Height of window.))));

    // Parse the program arguments.
    auto result = cli.parse({ argc, argv });

    if (get_help)
    {
        std::cout << cli;
        return 0;
    }

    // Check that the arguments where valid.
    if (!result)
    {
        std::cerr << "Error in command line: " << result.message()
            << std::endl;
        return 1;
    }

    if (show_full_screen)
    {
        std::cout << "Full screen aspect = " << aspect << "\n";
    }
    else
```

```
{  
    std::cout << "Window size = " << width << " x " << height << "\n";  
}  
  
return 0;  
}
```

- 1 The `ratio` option is a regular value argument.
- 2 The group for the window size turns off the full screen mode when the size options are fully specified.
- 3 We add the `width` and `height` options to the group.
- 4 We specify both the `width` and `height` as required to ensure we get only a valid specification when both are present. Otherwise the group is ignored as optional.

8. Sub-commands

A common program pattern is to have a single "shell" program that performs various independent commands. For example you have `git commit..`, `git push..`, and so on. It's possible to create such sub-command parsers in Lyra with the use of `arguments`, `group` and `literal` parsers manually. But as this is a very common use case the library provides a convenience group parser for the occasion.

In this example we use two `command` parameters to specify the sub-commands and a lambda to execute the subcommands.

The parsing for this example is arranged into two groups. One for global options and another for the commands. This follows the common practice of accepting options then commands in this scenario. It also allows us to specify that we only allow a single command, by using a `require(1,1)` specification.

C++

```
#include <lyra/lyra.hpp>

#include <iostream>
#include <string>
#include <vector>

// Run a process, sub-command data.
struct run_command 1
{
    std::vector<std::string> command; 2
    bool verbose = false;
    bool show_help = false;

    void add_command(lyra::group & g) 3
    {
        g.add_argument(lyra::command("run",
            [this](const lyra::group & f) { this->do_command(f); }) 4
            .help("Execute the given command.")
            .add_argument(lyra::help(show_help))
            .add_argument(lyra::opt(verbose)
                .name("-v")
                .name("--verbose")
                .optional()
                .help(
                    "Show additional output as to what we are doing."))
            .add_argument(lyra::arg(command, "command")
                .required()
                .help(
                    "The command, and arguments, to attempt to run.")))));
    }

    void do_command(const lyra::group & g)
    {
        if (show_help)
            std::cout << g; 5
        else
        {
            std::cout << "RUN: "
                << "verbose=" << (verbose ? "true" : "false");
            for (auto c : command) std::cout << " " << c;
            std::cout << "\n";
        }
    }
};

// Kill a named process, sub-command data.
struct kill_command 6
{
    std::string process_name;
    int signal = 9;
    bool show_help = false;

    void add_command(lyra::group & g)
    {
        g.add_argument(lyra::command(
            "kill", [this](const lyra::group & f) { this->do_command(f); })
            .help("Terminate the process with the given name.")
            .add_argument(lyra::help(show_help))
```

```

        .add_argument(lyra::opt(signal, "signal")
            .name("-s")
            .name("--signal")
            .optional()
            .help(
                "The signal integer to post to the running process."))
        .add_argument(lyra::arg(process_name, "process_name")
            .required()
            .help(
                "The name of the process to search and signal.));
    }

void do_command(const lyra::group & g)
{
    if (show_help)
        std::cout << g;
    else
        std::cout << "KILL:"
            << " signal=" << signal << " process=" << process_name
            << "\n";
}

};

int main(int argc, const char ** argv)
{
    lyra::group global; 7
    bool show_help = false;
    global.add_argument(lyra::help(show_help));

    lyra::group subcommands; 8
    subcommands.require(1, 1); 9
    kill_command kill;
    kill.add_command(subcommands);
    run_command run;
    run.add_command(subcommands);

    auto cli = lyra::cli().add_argument(global).add_argument(subcommands);

    auto result = cli.parse({ argc, argv }); 10
    if (show_help)
    {
        std::cout << cli;
        return 0;
    }
    if (!result) 11
    {
        std::cerr << result.message() << "\n";
    }
    return result ? 0 : 1;
}

```

- 1 A simple `struct` for information on the sub-commands. First for our `run` sub-command.
- 2 The arguments for the sub-command.
- 3 The constructor defines the additional arguments for the sub-command in the given `cli`.
- 4 Each sub-command sets a callback for when the group is successfully parsed which tells us we have a valid command to respond to.

- 5 We specified a sub-command specific help option. Here we can check for it and print out the help for the `(group)` only. This help will look similar to the full help output, but only contains the `(group)` arguments.
- 6 And now the information for our `(kill)` sub-command.
- 7 A group of global options.
- 8 A group of the accepted commands.
- 9 The group accepts exactly one command.
- 10 We go ahead and parse the top-level `(cli)` which will also parse the sub-command groups.
- 11 At the end we can do the regular error handling.

9. Main

For many cases specifying a help option, handling it, and specifying additional arguments introduces boilerplate that can get in the way of managing "easy" command lines. The `(lyra::main)` class abstract most of that boilerplate to make it trivial to create and handle a command line.

In this example we define and use two options and one positional argument. It then executes the lambda with the parsed command line as given by `(argv)`.

```
#include <iostream>
#include <lyra/lyra.hpp>

int main(int argc, const char ** argv)
{
    return lyra::main() 1
        ("-x", 0)("-y", 0)("z", 5) 2
        (argc, argv, [] (lyra::main & m) { 3
            std::cout << (int(m["-x"]) + int(m["-y"])) * int(m["z"]) << "\n"; 4
            return 0; 5
        });
}
```

C++

- 1 Use the `(lyra::main)` utility by creating an instance of it.
- 2 Define two options and one positional argument. Each one defines the one option, or hint, and the default value (and type of value).
- 3 Define and execute the body of the program passed in as a lambda.
- 4 Access the individual options and argument from any of their names or hint. Each is converted to the specified type (`(int)` in this case).
- 5 Return an appropriate value that is also return from `(main)` entry function.

10. More Main

Using the simple `(lyra::main)` interface can't possibly cover the variety of use cases people want. In particular when it comes to the options and arguments. For that the `(lyra::main)` interface supports adding instances of the regular option and argument parsers.

In this example we define and use two options and one positional argument, as before. But we use `(lyra::opt)`

and `lyra::arg` specifications. We also use the `lyra::val` value holder to provide the location for the parsed value and it's parsed type.

```
#include <iostream>
#include <lyra/lyra.hpp>

int main(int argc, const char ** argv)
{
    return lyra::main()
        (lyra::opt(lyra::val(0), "x")["-x"]) 1
        (lyra::opt(lyra::val(0), "y")["-y"])
        (lyra::arg(lyra::val(5), "z")) 2
        (argc, argv, [](lyra::main & m)
        {
            std::cout << (int(m["-x"]) + int(m["-y"]))*int(m["z"]) << "\n";
            return 0;
        });
}
```

C++

- 1 Add the "-x" option as an integer with a default of 0.
- 2 The positional argument in this case has a default of 5.

11. Counting Flags

Some times we want to allow multiple flags to be specified. This is most common with a verbose (`-v`) option. For example in passing in `-vvv` to indicate more verbose output from a program than using `-v`. To do this we can specify the cardinality as usual, and make use of passing in a lambda for the value of the option instead of a variable reference. With the lambda we can count the number of the flag occurrences.

```
#include <iostream>
#include <lyra/lyra.hpp>

int main(int argc, const char ** argv)
{
    int verbose = 0; 1
    return lyra::main()
        (lyra::opt(
            [&](bool){ verbose += 1; }) 2
            ["-v"].cardinality(0,5)) 3
        (argc, argv, [&](lyra::main &)
        {
            std::cout << verbose << "\n"; 4
            return 0;
        });
}
```

C++

- 1 We start off the verbose as "off" by having zero count.
- 2 The option takes a lambda for the value that counts the flag each time the user specifies it.
- 3 We allow a verbose level up to 5, i.e. five `-v` flags can be specified.
- 4 We end up with the counted `verbose` level after parsing.

12. Option Styles

This library, by default, uses the style of specifying options most commonly seen in POSIX flavored systems like Linux and BSD of having long options prefixed with double dash (`--`) where the value follow an equal (`=`). And short options, i.e. flags, prefixed with a single dash (`-`). This is not, by far, the only variation of option style in the wild. To accommodate such style variations one can use the `lyra::option_style` class to define what one wants the user to use. The `lyra::option_style` is independent of the syntax used to specify the options for parsing. And hence the same argument option definition can be used to parse different option styles.

There are three aspects of the options that can be configured in the `lyra::option_style`:

`value_delimiters`

The characters between an option and its value.

`long_option_prefix` and `long_option_size`

The characters for long option name prefix, and how many repetitions of the character.

`short_option_prefix` and `short_option_size`

The characters for short option name prefix, and how many repetitions of the character.

To specify the default POSIX style one would specify it with these settings:

`value_delimiters` `=` and space

`long_option_prefix` `-`

`long_option_size` `2`

`short_option_prefix` `-`

`short_option_size` `1`

There are some built-in option Styles defined for common cases returned by class methods. But you can define, and use your own using the `lyra::cli::style` method. For example:

C++


```

#include <iostream>
#include <lyra/lyra.hpp>

int main(int argc, const char ** argv)
{
    using namespace lyra;

    bool show_help = false;
    std::string name;
    bool flag = false;
    auto cli
        = help(show_help).description("This is a sample CLI parser.") 1
        | opt(name, "name")["-n"]["--name"]("The name to use.") 2
        | opt(flag)["-f"]["--flag"]("A flag to set.");

    cli.style({ ",", "+", 2, "~", 1 }); 3

    auto result = cli.parse({ argc, argv });
    if (!cli.parse({ argc, argv }))
    {
        std::cerr << result.message() << "\n\n";
    }
    if (show_help || !result)
    {
        std::cout << cli << "\n";
    }
    return result ? 0 : 1;
}

```

- 1 The help option will add `(-h)`, `(-?)`, and `(--help)`.
- 2 We add other options, `(-n)` and `(--name)`.
- 3 Our style accepts arguments like: `(++name, Jane)`, `(~f)`, and `(~?)`.

13. Tail Arguments

A common pattern for command line programs is to use "escaped" arguments at the end of the command. Usually with an empty option prefix as the escape indicator. For example: `command --option - arbitrary arguments --more`. For which the `(--option)` is treated as a regular option. But everything after the `(--)` is immune to parsing for options. Such behaviour can be done in Lyra with a combination of a group, a literal, and a container to capture the rest of the arguments.

This example shows how you could do that. Do note that as one can create multiple Lyra parsers. It would also be possible to use an additional parser to also parse the escaped argument differently.

C++

```

#include <iostream>
#include <lyra/lyra.hpp>
#include <string>
#include <vector>

int main(int argc, const char ** argv)
{
    bool echo = false;
    bool help = false;
    std::vector<std::string> rest;

    auto cli = lyra::cli();
    cli.add_argument(lyra::help(help));
    cli.add_argument(lyra::opt(echo).name("--echo").help(
        "Echo the post options arguments."));
    cli.add_argument(lyra::group()
        .add_argument(lyra::literal("--"))
        .add_argument(lyra::arg(rest, "rest")));

    auto result = cli.parse({ argc, argv });

    if (!result)
    {
        std::cerr << "Error in command line: " << result.message() << "\n";
        return 1;
    }

    if (help)
    {
        std::cout << cli << "\n";
        return 0;
    }

    if (echo)
    {
        std::cout << "rest:";
        for (auto & r : rest) std::cout << " " << r;
        std::cout << "\n";
    }
    return 0;
}

```

14. Reference

14.1. `lyra::arg`

A parser for regular arguments, i.e. not `--` or `-` prefixed. This is simply a way to get values of arguments directly specified in the cli.

Is-a `lyra::bound_parser`.

Construction

```
template <typename Reference>
arg<Derived>::arg(Reference& ref, std::string const& hint);

template <typename Lambda>
arg<Derived>::arg(Lambda const& ref, std::string const& hint);

template <typename Lambda>
arg<Derived>::arg(Lambda && ref, std::string const& hint);
```

Constructs a value argument with a target typed variable or callback. These are bare arguments that take a value as in `(value)`. In the first form the given `(ref)` receives the value of the argument after parsing. The second form the callback is invoked during the parse with the given value. Both take a `(hint)` that is used in the help text. When the argument can be specified multiple times the callback will be called consecutively for each argument value given. And if a container is given as a reference on the first form it will contain all the specified values.

14.2. `lyra::args`

Transport for raw args (copied from main args, supplied via init list, or from a pair of iterators).

14.3. `lyra::arguments`

A combined parser made up of any number of sub-parsers. Creating and using one of these as a basis, one can incrementally compose other parsers into this one. For example:

```
auto p = lyra::arguments();
std::string what;
float when = 0;
std::string where;
p |= lyra::opt(what, "what")["--make-it-so"]("Make it so.").required();
p |= lyra::opt(when, "when")["--time"]("When to do <what>.").optional();
p.add_argument(lyra::opt(where, "where").name("--where")
    .help("There you are.").optional());
```

C++

Construction

Default

```
arguments() = default;
```

C++

Default constructing a `(arguments)` is the starting point to adding arguments and options for parsing a arguments line.

Copy

```
arguments::arguments(const arguments& other);
```

C++

Specification

Composition

This parser is composed of sub-parsers that consume the arguments as possible. One can configure how the sub-parsers are used. And depending on the mode the ordering of the sub-parsers can matter.

lyra::arguments::add_argument

```
arguments& arguments::add_argument(parser const& p);  
arguments& arguments::operator|=(parser const& p);  
arguments& arguments::add_argument(arguments const& other);  
arguments& arguments::operator|=(arguments const& other);
```

C++

Adds the given argument parser to the considered arguments for this `arguments`. Depending on the parser given it will be: directly added as an argument (for `parser`), or add the parsers from another `arguments` to this one.

Parsing Mode

The parsing mode controls how the parsing of the added arguments happens. Depending on how this is specified different parsing algorithms get used to match parsers with arguments.

lyra::arguments::sequential

```
arguments & arguments::sequential();
```

C++

Sets the parsing mode for the arguments to "sequential". When parsing the arguments they will be, greedily, consumed in the order they were added. This is useful for sub-commands and structured command lines.

lyra::arguments::inclusive

```
arguments & arguments::inclusive();
```

C++

Sets the parsing mode for the arguments to "inclusively any". This is the default that attempts to match each parsed argument with all the available parsers. This means that there is no ordering enforced.

lyra::arguments::relaxed

```
arguments & arguments::relaxed();
```

C++

Sets the parsing mode for the arguments to "relaxed any". When parsing the arguments it attempts to match each parsed argument with all the available parsers. This means that there is no ordering enforced. Unknown, i.e. failed, parsing are ignored.

Limits

The parsing of sub-parsers can be restrained with limits. The limits can control how many parsed arguments are required or allowed.

lyra::arguments::require

```
arguments & arguments::require(std::size_t n, std::size_t m);
```

C++

Requires a minimum and/or maximum number of arguments **only** to be successfully parsed by the sub-parsers to make this collection of arguments valid. Specifying the minimum or maximum as zero (`0`) indicates it's undefined. I.e. there would be no minimum or maximum. If a maximum is indicated, as soon as that maximum is reached the parsing is considered successful and completes.



Side-effects of parsing arguments, even if overall the collection as a whole fails for not satisfying the required bounds.

Other

`lyra::arguments::get`

```
template <typename T>
T & arguments::get(std::size_t i);
```

C++

Get a modifiable reference to one of the parsers specified.

`lyra::operator<<`

```
template <typename T>
T & operator<<(T & os, arguments const & a);
```

C++

Prints the help text for the arguments to the given stream `os`. The `os` stream is not used directly for printing out. Instead a `lyra::printer` object is created by calling `lyra::make_printer(os)`. This indirection allows one to customize how the output is generated.

14.4. `lyra::cli`

A Combined parser made up of any two or more other parsers. Creating and using one of these as a basis one can incrementally compose other parsers into this one. For example:

```
auto cli = lyra::cli();
std::string what;
float when = 0;
std::string where;
cli |= lyra::opt(what, "what")["--make-it-so"]("Make it so.").required();
cli |= lyra::opt(when, "when")["--time"]("When to do <what>.").optional();
cli.add_argument(lyra::opt(where, "where").name("--where")
    .help("There you are.").optional());
```

C++

Construction

Default

```
cli() = default;
```

C++

Default constructing a `cli` is the starting point to adding arguments and options for parsing a command line.

Copy

```
cli::cli(const cli& other);
```

C++

Specification

`lyra::cli::add_argument`

C++

```
cli& cli::add_argument(exe_name const& exe_name);
cli& cli::operator|=(exe_name const& exe_name);
cli& cli::add_argument(parser const& p);
cli& cli::operator|=(parser const& p);
cli& cli::add_argument(group const& p);
cli& cli::operator|=(group const& p);
cli& cli::add_argument(cli const& other);
cli& cli::operator|=(cli const& other);
```

Adds the given argument parser to the considered arguments for this `cli`. Depending on the parser given it will be: recorded as the exe name (for `exe_name` parser), directly added as an argument (for `parser`), or add the parsers from another `cli` to this one.

`lyra::cli::operator[]`

```
cli::value_result cli::operator[](const std::string & n)
```

C++

Finds the given argument by either option name or hint name and returns a convertible reference to the value, either the one provided by the user or the default.

`lyra::cli::parse`

```
parse_result cli::parse(
    args const& args, const option_style& customize) const;
```

C++

Parses given arguments `args` and optional option style. The result indicates success or failure, and if failure what kind of failure it was. The state of variables bound to options is unspecified and any bound callbacks may have been called.

`lyra::cli::style`

```
lyra::cli & lyra::cli::style(const lyra::option_style & style)
lyra::cli & lyra::cli::style(lyra::option_style && style)
```

C++

Specifies the `lyra::option_style` to accept for this instance.

`lyra::cli::style_print_short_first`

```
lyra::cli & lyra::cli::style_print_short_first()
```

C++

Specifies print options sorted with short options appearing first.

`lyra::cli::style_print_long_first`

```
lyra::cli & lyra::cli::style_print_long_first()
```

C++

Specifies print options sorted with long options appearing first.

`lyra::operator<<`

C++

```
template <typename T>
T & operator<<(T & os, cli const & c);
```

Prints the help text for the cli to the given stream `os`. The `os` stream is not used directly for printing out. Instead a `lyra::printer` object is created by calling `lyra::make_printer(os)`. This indirection allows one to customize how the output is generated.

14.5. `lyra::command`

A parser that encapsulates the pattern of parsing sub-commands. It provides a quick wrapper for the equivalent arrangement of `group` and `literal` parsers. For example:

```
lyra::command c = lyra::command("sub");
```

C++

Is equivalent to:

```
lyra::command c = lyra::group()
    .sequential()
    .add_argument(literal("sub"))
    .add_argument(group());
lyra::group & g = c.get<lyra::group>(1);
```

C++

I.e. it's composed of a `literal` followed by the rest of the command arguments.

Is-a `lyra::group`.

Construction

```
command::command(const std::string & n);
command::command(
    const std::string & n, const std::function<void(const group &)>& f);
```

C++

To construct an `command` we need a name `(n)` that matches, and triggers, that command.

Specification

`lyra::command::help`

```
command & command::help(const std::string& text)
command & command::operator()(std::string const& description)
```

C++

Specify a help description for the command. This sets the help for the underlying literal of the command.

`lyra::command::add_argument`

```
template <typename P>
command & command::add_argument(P const & p);
template <typename P>
command & command::operator|=(P const & p);
```

C++

Adds the given argument parser to the considered arguments for this `command`. The argument is added to the sub-group argument instead of this one. Hence it has the effect of adding arguments **after** the command name.

`lyra::command::brief_help`

```
command & command::brief_help(bool brief = true);
```

C++

Enables, or disables with `false`, brief output of the top level help. Brief output only prints out the command name and description for the top level help (i.e. `std::cout << cli`). You can output the full command, options, and arguments by printing the command (i.e. `std::cout << command`).

14.6. `lyra::exe_name`

Specifies the name of the executable.

Is-a `lyra::composable_parser`.

Construction

```
exe_name::exe_name(std::string& ref)
```

C++

Constructs with a target string to receive the name of the executable. When the `cli` is run the target string will contain the exec name.

```
template <typename LambdaT>
exe_name::exe_name(LambdaT const& lambda)
```

C++

Construct with a callback that is called with the value of the executable name when the `cli` runs.

Accessors

`lyra::exe_name::name`

```
std::string exe_name::name() const
```

C++

Returns the executable name when available. Otherwise it returns a default value.

`lyra::exe_name::set`

```
parser_result exe_name::set(std::string const& newName)
```

C++

Sets the executable name with the `newName` value. The new value is reflected in the bound string reference or callback.

14.7. `lyra::group`

A group of arguments provides for parsing, optionally, a set of arguments together. The group itself is considered successfully parsed only when the arguments in the group are parsed without errors. A common use case for this are sub-commands. This implementation is recursive. And hence allows groups within groups for

describing branching argument parsing.

Is-a `lyra::arguments`.

Construction

Default

```
group();
```

C++

Default constructing a `group` does not register the success callback.

Copy

```
group::group(const group & other);
```

C++

Success Handler

```
group::group(const std::function<void(const group &)> & f)
```

C++

Registers a function to call when the group is successfully parsed. The function is called with the group to facilitate customization.

`lyra::group::optional`

```
group & group::optional();
```

C++

Indicates that the argument is optional. This is equivalent to specifying `cardinality(0, 1)`.

`lyra::group::required(n)`

```
group & group::required(std::size_t n);
```

C++

Specifies that the argument needs to given the number of `n` times (defaults to 1).

`lyra::group::cardinality(n)`

```
group & group::cardinality(std::size_t n);
group & group::cardinality(std::size_t n, std::size_t m);
```

C++

Specifies the number of times the argument can and needs to appear in the list of arguments. In the first form the argument can appear exactly `n` times. In the second form it specifies that the argument can appear from `n` to `m` times inclusive.

14.8. `lyra::help`

Utility function that defines a default `--help` option. You can specify a `bool` flag to indicate if the help option was specified and that you could display a help message.

The option accepts `-?`, `-h`, and `--help` as allowed option names.

```
help & help::description(const std::string &text)
```

C++

Sets the given `(text)` as the general description to show with the help and usage output for CLI parser. This text is displayed between the "Usage" and "Options, arguments" sections.

14.9. `lyra::literal`

A parser that matches a single fixed value.

Is-a `lyra::parser`.

Construction

Token

```
inline literal::literal(std::string const& n)
```

C++

Constructs the literal with the name of the token to match.

Specification

`lyra::literal::help`

```
literal& literal::help(const std::string& help_description_text)
literal& literal::operator()(std::string const& help_description_text)
```

C++

Specify a help description for the literal.

14.10. `lyra::main`

Encapsulates the common use case of a main program that has a help option and has a minimal way to specify and parse options. This provides for a way to specify options and arguments in a simple function form. It handles checking for errors and reporting problems.

Construction

```
main::main(const std::string & text);
```

C++

Construct with text for description, which defaults to an empty string. The description is specified for the help option that is added to the command line.

Add Argument

```
template <typename T> main & main::operator()(const T & arg_parser)
template <typename T> main & main::add_argument(const T & arg_parser)
template <typename T> main & main::operator|=(const T & arg_parser)
```

C++

Adds a parser as an argument to the command line. These forward directly to the `lyra::cli` equivalents. The added parser can be any of the regular Lyra parsers like `lyra::opt` or `lyra::arg`.

Simple Args

```
template <typename V>
main & main::operator()(
    std::initializer_list<std::string> arg_names, V && default_value)
template <typename V>
main & main::operator()(const std::string & arg_name, V && default_value)
```

Specifies, and adds, a new argument. Depending on the `arg_names` it can be either a `lyra::opt` or `lyra::arg`. The first item in `arg_names` indicates the type of argument created and added:

Specify either `-<name>` or `--<name>` to add a `lyra::opt`. You can specify as many option names following the first name. A name that doesn't follow the option syntax is considered as the help text for the option.

Specify a non `(-)` prefixed name as the first item to signify a positional `lyra::arg`.

The single `std::string` call is equivalent to specifying just the one option or argument.

Example specifications:

<code>("-o", 0)</code>	Short <code>(-o)</code> option as <code>(int)</code> value.
<code>("--opt", 0)</code>	Long <code>--opt</code> option as <code>(int)</code> value.
<code>({"-o", "--opt"}, 1.0f)</code>	Short and long option as <code>(float)</code> value.
<code>({"-o", "The option."}, 1.0f)</code>	Short option and help description as <code>(float)</code> value.
<code>("opt", 2)</code>	Positional, i.e. <code>lyra::arg</code> , argument as <code>(int)</code> value.
<code>({"opt", "The option."}, 2)</code>	Positional argument and help description as <code>(int)</code> value.
<code>("--opt", std::vector<float>())</code>	Long option with as multiple float values.

Execute

```
template <typename L>
int main::operator()(const args & argv, L action)
template <typename L>
int main::operator()(int argc, const char ** argv, L action)
```

Executes the given action after parsing of the program input arguments. It returns either `0` or `1` if the execution was successful or failed respectively. The `action` is called with the `lyra::main` instance to provide access to the parsed argument values.

14.11. `lyra::opt`

A parser for one option with multiple possible names. The option value(s) are communicated through a reference to a variable, a container, or a callback.

Is-a `lyra::bound_parser`.

Construction

Flags

```
lyra::opt::opt(bool& ref);

template <typename L>
lyra::opt::opt(L const& ref);

template <typename L>
lyra::opt::opt(L && ref);
```

C++

Constructs a flag option with a target `(bool)` to indicate if the flag is present. The first form takes a reference to a variable to receive the `(bool)`. The second takes a callback that is called with `(true)` when the option is present.

Values

```
template <typename T>
lyra::opt::opt(T& ref, std::string const& hint);

template <typename L>
lyra::opt::opt(L const& ref, std::string const& hint)

template <typename L>
lyra::opt::opt(L && ref, std::string const& hint)
```

C++

Constructs a value option with a target `(ref)`. The first form takes a reference to a variable to receive the value. The second takes a callback that is called with the value when the option is present.

Specification

`lyra::opt::name`

```
lyra::opt& lyra::opt::name(const std::string &opt_name)
lyra::opt& lyra::opt::operator[](const std::string &opt_name)
```

C++

Add a spelling for the option of the form `(--<name>)` or `(-n)`. One can add multiple short spellings at once with `(-abc)`.

14.12. `lyra::option_style`

Specify the syntax style for options to the parser.

```
std::string value_delimiters;
std::string long_option_prefix;
std::size_t long_option_size = 0;
std::string short_option_prefix;
std::size_t short_option_size = 0;
opt_print_order options_print_order = opt_print_order::per_declaration;
```

C++

- `(value_delimiters)` — Specifies a set of characters that are accepted as a delimiter/separator between an option name and an option value when a single argument is used for the option+value (i.e. "--option=value").

- `(long_option_prefix)`—Specifies a set of characters that are accepted as a prefix for long options (i.e. multi-char single option name).
- `(long_option_size)`—The number of prefix characters that indicates a long option. A value of zero (0) indicates that long options are not accepted.
- `(short_option_prefix)`—Specifies a set of characters that are accepted as a prefix for short options (i.e. single-char multi-options).
- `(short_option_size)`—The number of prefix characters that indicates a short option. A value of zero (0) indicates that short options are not accepted.
- `(options_print_order)`—The order to print the options section of the help text. Possible values: `(per_declaration)`, `(sorted_short_first)`, `(sorted_long_first)`.
- `(indent_size)`—The character count to indent argument detail description. This affects the indenting of usage detail, the arguments, and any sub-commands/arguments recursively. The default is "2".

Construction

```
lyra::option_style::option_style(
    std::string && value_delimiters,
    std::string && long_option_prefix = {},
    std::size_t long_option_size = 0,
    std::string && short_option_prefix = {},
    std::size_t short_option_size = 0)
```

C++

Utility constructor that defines all the settings.

Definitions

```
std::string lyra::option_style::long_option_string() const
std::string lyra::option_style::short_option_string() const
```

C++

Gives the default long or short option string, or prefix, for this option style. If the type of option is not available, i.e. size is zero, an empty string is returned.

Styles

```
static const option_style & lyra::option_style::posix();
static const option_style & lyra::option_style::posix_brief();
static const option_style & lyra::option_style::windows();
```

C++

These provide definitions for common syntax of option styles:

`(posix)`

The overall *default* that is two dashes (`(--)`) for long option names and one dash (`(-)`) for short option names. Values for long options use equal (`(=)`) between the option and value.

`(posix_brief)`

Variant that only allows for long option names with a single dash (`(-)`).

windows

The common option style on Windows (**CMD.EXE**) shell. It only allows long name options that start with slash (**/**) where the value is specified after a colon (**:**). Single character flag style options are only available as individual long options, for example (**/A**).

14.13. lyra::parser_result

The result of parsing arguments.

14.14. lyra::parser

Base for all argument parser types.

Specification**lyra::parser::get_usage_text**

```
virtual std::string get_usage_text(const option_style &) const;
```

C++

Returns the formatted (**USAGE**) text for this parser, and any contained sub-parsers. This is called to print out the help text from the stream operator.

lyra::parser::get_description_text

```
virtual std::string get_description_text(const option_style &) const;
```

C++

Returns the description text for this, and any contained sub-parsers. This is called to print out the help text from the stream operator.

14.15. lyra::composable_parser

A parser that can be composed with other parsers using (**operator|**). Composing two (**composable_parser**) instances generates a (**cli**) parser.

14.16. lyra::bound_parser

Parser that binds a variable reference or callback to the value of an argument.

Construction

```
template <typename Derived>
template <typename Reference>
bound_parser<Derived>::bound_parser(Reference& ref, std::string const& hint);
```

C++

```
template <typename Derived>
template <typename Lambda>
bound_parser<Derived>::bound_parser(Lambda const& ref, std::string const& hint);
```

```
template <typename Derived>
template <typename Lambda>
bound_parser<Derived>::bound_parser(Lambda && ref, std::string const& hint);
```

Constructs a value option with a target typed variable or callback. These are options that take a value as in `--opt=value`. In the first form the given `(ref)` receives the value of the option after parsing. The second form the callback is called during the parse with the given value. Both take a `(hint)` that is used in the help text. When the option can be specified multiple times the callback will be called consecutively for each option value given. And if a container is given as a reference on the first form it will contain all the specified values.

Specification

`lyra::bound_parser::help`, `lyra::bound_parser::operator(help)`

```
template <typename Derived>
Derived& bound_parser<Derived>::help(std::string const& help_description_text);
template <typename Derived>
Derived& bound_parser<Derived>::operator()(std::string const&
help_description_text);
```

C++

Defines the help description of an argument.

`lyra::bound_parser::optional`

```
template <typename Derived>
Derived& bound_parser<Derived>::optional();
```

C++

Indicates that the argument is optional. This is equivalent to specifying `cardinality(0, 1)`.

`lyra::bound_parser::required(n)`

```
template <typename Derived>
Derived& bound_parser<Derived>::required(std::size_t n);
```

C++

Specifies that the argument needs to given the number of `(n)` times (defaults to 1).

`lyra::bound_parser::cardinality(n, m)`

```
template <typename Derived>
Derived& bound_parser<Derived>::cardinality(std::size_t n);

template <typename Derived>
Derived& bound_parser<Derived>::cardinality(std::size_t n, std::size_t m);
```

C++

Specifies the number of times the argument can and needs to appear in the list of arguments. In the first form the argument can appear exactly `(n)` times. In the second form it specifies that the argument can appear from `(n)` to `(m)` times inclusive.

`lyra::bound_parser::choices`

C++

```
template <typename Derived>
template <typename T, typename... Rest>
lyra::opt& lyra::bound_parser<Derived>::choices(T val0, Rest... rest)

template <typename Derived>
template <typename Lambda>
lyra::opt& lyra::bound_parser<Derived>::choices(Lambda const &check_choice)
```

Limit the allowed values of an argument. In the first form the value is limited to the ones listed in the call (one or more values). In the second form the `check_choice` function is called with the parsed value and returns `true` if it's an allowed value.

`lyra::bound_parser::hint`

```
template <typename Derived>
std::string lyra::bound_parser<Derived>::hint() const

template <typename Derived>
Derived & lyra::bound_parser<Derived>::hint(std::string const & hint)
```

C++

Selectors to read and write the hint of a variable-bound parser.

The hint should not be modified anymore, once the parser is applied to arguments or used in a

`lyra::composable_parser`.

14.17. `lyra::printer`

A `printer` is an interface to manage formatting for output. Mostly the output is for the help text of lyra cli parsers. The interface abstracts away specifics of the output device and any visual arrangement, i.e. padding, coloring, etc.

```
virtual printer & printer::heading(
    const option_style & style,
    const std::string & txt) = 0;
virtual printer & printer::paragraph(
    const option_style & style,
    const std::string & txt) = 0;
virtual printer & printer::option(
    const option_style & style,
    const std::string & opt,
    const std::string & description) = 0;
```

C++

Indenting levels is implemented at the base and the indent is available for concrete implementations to apply as needed.

```
virtual printer & indent(int levels = 1);
virtual printer & dedent(int levels = 1);
virtual int get_indent_level() const;
```

C++

You can customize the printing output by implementing a subclass of `lyra::printer` and implementing a

corresponding `make_printer` factory function which matches the output to the printer. For example:

```
inline std::unique_ptr<my_printer> make_printer(my_output & os_)
{
    return std::unique_ptr<my_printer>(new my_output(os_));
}
```

C++

14.18. `lyra::ostream_printer`

A `lyra::printer` that uses `std::ostream` for output. This is the one used in the case of printing to the standard output channels of `std::cout` and `std::cerr`.

14.19. `lyra::val`

```
auto val(T && v);
auto val(const char * v);
```

C++

Makes a bound self-contained value of the type of the given r-value. The created bound values can be used in place of the value references for arguments. And can be retrieved with the `lyra::cli::operator[]` call.

15. History

15.1. 1.8

Changes:

- Fix cmake version for deprecation of 3.5.— *Tomáš Slanina*
- Remove some unused code in type traits compatibility code.— *Paolo Pastori*
- Fix short option not using the option style for extracting value.— *Paolo Pastori*
- Fix spec of help for arguments in main style help API.— *Paolo Pastori*
- Fix various warnings in the most pedantic compile modes.— *René Ferdinand Rivera Morell*
- Add example for option termination with arbitrary tail arguments.— *René Ferdinand Rivera Morell*
- Fix short-option with value not being recognized when part of other short options.— *René Ferdinand Rivera Morell*
- Fix error message when a required argument is not matched.— *René Ferdinand Rivera Morell*
- Fix error message for sequence parsed arguments.— *René Ferdinand Rivera Morell*
- Improve error message for parsing arguments in group, like commands, to better reflect user intent.— *René Ferdinand Rivera Morell*

15.2. 1.7

It's been a rather long time since a release. But hopefully releases will be more frequent from now on because.. Instead of waiting for a "large enough" accumulation of changes in a release the plan going forward is to have at least one release a year.

In this release there are a variety of bug fixes and adjustments. Although most are small they do add up to a good chunk of new stability. There are also some enhancements as requested that make for more flexibility in output and parsing modes.

Changes:

- **New:** Make printing help text configurable to make it possible to avoid using iostreams for output. — *René Ferdinand Rivera Morell*
- **New:** Implements a new `(brief_help)` mode for a `(command)` that only prints out the command name and description. — *René Ferdinand Rivera Morell*
- **New:** Adds printing default/initial value for options in help output. — *René Ferdinand Rivera Morell*
- **New:** Add `(relaxed)` parsing to support "bag of options" style command lines. — *René Ferdinand Rivera Morell*
- **New:** Add ordering/sorting of printed option details specified in the `(option_style::options_print_order)` value. — *René Ferdinand Rivera Morell*
- **New:** Improve output readability by indenting sub-commands. — *René Ferdinand Rivera Morell*
- **New:** Add `(lyra::arguments::require)` to specify limits on how many successful sub-parsers are required for parsing. — *René Ferdinand Rivera Morell*
- **Fix** `(operator|)` loosing group subtype causing invalid parsing. — *René Ferdinand Rivera Morell*
- **Fix** crash when no arguments are given to the parsing. — *René Ferdinand Rivera Morell*
- **Fix** crash when changing settings of a command and adding subsequent arguments. — *René Ferdinand Rivera Morell*
- **Fix** short option values being ignored in the case where the value separator is past the short option and in the value itself. — *René Ferdinand Rivera Morell*
- **Fix** choice limited args from erroring early. — *René Ferdinand Rivera Morell*
- **Refactored** the way help is printed to allow individual parsers to customize their output through the `(printer)` object. — *René Ferdinand Rivera Morell*
- **Fix** zero-as-null pointer conversion warnings. — *Hermann von Kleist*
- **Fix** some unclear error messages when parsing optional arguments. — *René Ferdinand Rivera Morell*
- **Fix** parsing with mix of required and optional arguments. — *René Ferdinand Rivera Morell*
- **Fix** passing move-only lambdas to `(opt)` and `(arg)` parsers. — *René Ferdinand Rivera Morell*
- **Fix** incorrect interpretation of single character literal strings to the `(choice(...))` function. — *René Ferdinand Rivera Morell*
- **Fix** ignored short circuit error results returned from user callbacks. This now, correctly, allows for custom error messages from user callbacks when parsing or checking arguments. — *René Ferdinand Rivera Morell*
- **Tested** with Visual Studio 2022, MinGW-64 (gcc 8.1.0), Linux (clang 3.9, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21; gcc 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16), Xcode (14.3.1, 15.4, 16.4), on Azure Pipelines; and with VS 2015, VS 2017, VS 2019, VS 2022, MinGW-64 (gcc 6.3, 7.3, 8.1) on AppVeyor.

15.3. 1.6.1

Only bug fixes for compile warnings and a pair of bad argument names.

Changes:

- Fix variable name shadowing warnings.— *Sadie Powell*
- Fix `command::operator|` function signature.— *Henrik Gaßmann*

15.4.1.6

Even though most of the changes for this release are to fix various reported bug fixes, fixing those bugs has meant dealing with some long standing internal parser inconsistencies. While I try to not break backwards compatibility, I likely can't catch all of the corner cases people have discovered.



This release has some internal cleanup of the parser result types. It also has an externally visible change to the `parse` function result type. It renames the `errorMessage()` function to `message()`. The `errorMessage()` function is marked deprecated and will be removed in a future release.

Changes:

- **New:** Add `lyra::option_style` type to better customize the syntax of options allowed. This replaces the "old" `lyra::parser_customization` as a better way. The `lyra::parser_customization` is now deprecated, and will be removed at some future release.— *René Ferdinand Rivera Morell*
- **New:** Add example to show how to count flags with cardinality greater than one (1). Shows how to handle increasing verbosity with `-vvvv`.— *René Ferdinand Rivera Morell*
- **New:** Add capability to control the cardinality of argument groups.— *René Ferdinand Rivera Morell*
- **New:** Allow setting hint after construction of `lyra::bound_parser`.— *Max Ferger*
- Fix virtual overload hiding of `parse` method.— *René Ferdinand Rivera Morell*
- Fix previously removed error for when expected option specifications do not have the required `-` prefix.— *René Ferdinand Rivera Morell*
- Fix inconsistencies in parsing arguments in a group followed by other arguments. (issue #49)— *René Ferdinand Rivera Morell*
- Fix incorrectly accepting negative values for unsigned integer arguments. The argument value conversion now checks the input for acceptable digits in the input before attempting to parse it.— *René Ferdinand Rivera Morell*
- Fix sub-command arguments being ignored even when they are marked as required.— *René Ferdinand Rivera Morell*
- Fix assigned values for flag options (`--flag=no`) being ignore. They now cause an error when specified.— *René Ferdinand Rivera Morell*
- Fix choices being allowed for for flag options, but ignored. Specifying choices on flag options now signal an error.— *René Ferdinand Rivera Morell*
- Tested with Visual Studio 2019, VS 2022, MinGW-64 (gcc 8.1.0), Linux (clang 3.9, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13; gcc 5, 6, 7, 8, 9, 10, 11), Xcode (11.2.1, 11.3.1, 11.4.1, 11.5, 11.6, 11.7, 12.0.1, 12.2, 12.3, 12.4, 13.0), on Azure Pipelines; and with VS 2015, VS

2017, MinGW-64 (gcc 6.3, 7.3) on AppVeyor.

15.5. 1.5.1

Minor change to bring back the missing option argument hints from the usage help output.

15.6. 1.5

This release has some big changes in easy of use and functionality. There are now alternate interfaces for specifying and fetching arguments. Also the help output is now more compact and clearer. There is also new support for sub-commands.



The `cli_parser` type was renamed to `cli`. Both the `cli_parser` type and the `lyra/cli_parser.hpp` header are deprecated and will be removed in a future release.



Changed the install CMake target from `BFG::Lyra` to `bfg::lyra`.

Changes:

- **New:** Main encapsulation of common use case of help option handling. With shorthand specification of arguments.— *René Ferdinand Rivera Morell*
- **New:** Ability to specify argument values, with default, without specifying a local external to the parser variable. Making it easier to specify command lines. The values can be retrieved from the `cli` as a map from argument names to the value.— *René Ferdinand Rivera Morell*
- **New:** Support construction of `lyra::args` from iterators for cases where the arguments come from some existing container.— *René Ferdinand Rivera Morell*
- **New:** Add Cmake install support.— *Jayesh Vinay Badwaik*
- **New:** Command argument to make it easy to specify sub-commands.— *René Ferdinand Rivera Morell*
- **New:** Alternative sequential argument parsing mode.— *René Ferdinand Rivera Morell*
- **New:** Grouped arguments to specify and handle sub-command parsers.— *René Ferdinand Rivera Morell*
- **New:** `literal` parser to directly match an argument.— *René Ferdinand Rivera Morell*
- Fix missing hint text for optional, `lyra::arg`, arguments.— *René Ferdinand Rivera Morell*
- Bring back some cleaner and compact formatting of the help text.— *René Ferdinand Rivera Morell*
- Tested with Visual Studio 2017, VS 2019, MinGW-64 (gcc 8.1.0), Linux (clang 3.9, 4, 5, 6, 7, 8, 9, 10; gcc 4.9, 5, 6, 7, 8, 9, 10), Xcode (11.1, 11.2.1, 11.3.1, 11.4.1, 11.5, 11.6), on Azure Pipelines; and with VS 2015, MinGW-64 (gcc 6.3, 7.3) on AppVeyor.

15.7. 1.4

Changes:

- **New:** Allow passing option value choices as an array.— *René Ferdinand Rivera Morell*
- Fix sample code in README to be correctly namespaced.— *Jayesh Badwaik*
- Fix commands example to actually show help when specified.— *Shreyas Potnis*

- Tested with Visual Studio 2017, VS 2019, MinGW-64 (gcc 8.1.0), Linux (clang 3.8, 3.9, 4, 5, 6, 7, 8, 9, 10; gcc 4.8, 4.9, 5, 6, 7, 8, 9, 10), Xcode (10.1, 10.2.1, 10.3, 11.1, 11.2.1, 11.3.1, 11.4), on Azure Pipelines; and with VS 2015, MinGW-64 (gcc 6.3, 7.3) on AppVeyor.

15.8.1.3

Changes:

- **New:** Accept `(-o2)` style option+value arguments. — *René Ferdinand Rivera Morell*
- **New:** Accept `(-abc)` option name spelling as shorthand for `(-a)`, `(-b)`, `(-c)`. — *René Ferdinand Rivera Morell*
- Fixed inconsistent handling of option values that look like options, like negative numbers. — *René Ferdinand Rivera Morell*
- Rewrote argument token parsing for clarity, to avoid extra token buffer allocation, and to allow for more option & value variations. — *René Ferdinand Rivera Morell*
- Fixed allowing junk extra characters after a non-string option value. — *René Ferdinand Rivera Morell*
- Support specifying a single value for choices of an argument. — *René Ferdinand Rivera Morell*
- Fix various problems with the commands example program. Also now the examples for the documentation are unit tested along with the regular unit tests. — *René Ferdinand Rivera Morell*
- Tested with Visual Studio 2015, VS 2017, VS 2019, MinGW-64 (gcc 8.1), Linux (clang 3.5, 3.6, 3.7, 3.8, 3.9, 4, 5, 6, 7, 8, 9; gcc 4.8, 4.9, 5, 6, 7, 8, 9), Xcode (9.0, 9.0.1, 9.1, 9.2, 9.3, 9.3.1, 9.4, 9.4.1, 10.0, 10.1, 10.2, 10.2.1, 10.3, 11.0, 11.1, 11.2, 11.2.1), on Azure Pipelines.

15.9.1.2

Changes:

- **New:** One can now accept 1-or-more on bound container arguments by only specifying `(required())` on such arguments. — *René Ferdinand Rivera Morell*
- **New:** One can now define a `(description)` text to display as general command information with the `(help::description)` method. — *René Ferdinand Rivera Morell*
- **New:** Add named methods as alternates for operators to add and specify arguments. They follow the familiar Python `(argparse)` nomenclature. — *René Ferdinand Rivera Morell*
- **New:** Single header file alternative version of the library. For those that need or want such a thing. — *René Ferdinand Rivera Morell*
- Improve help for arguments that specify a cardinality. — *René Ferdinand Rivera Morell*
- Addition of more complicated use cases to demonstrate flexibility of library. — *René Ferdinand Rivera Morell*
- Continued internal clean up of code for stability and easier future enhancements. — *René Ferdinand Rivera Morell*
- Tested with Visual Studio 2015, VS 2017, VS 2019, MinGW-64 (gcc 8.1), Linux (clang 3.5, 3.6, 3.7, 3.8, 3.9, 4, 5, 6, 7, 8, 9; gcc 4.8, 4.9, 5, 6, 7, 8, 9), Xcode (9.0, 9.0.1, 9.1, 9.2, 9.3, 9.3.1, 9.4, 9.4.1, 10.0, 10.1, 10.2, 10.2.1, 10.3, 11.0, 11.1, 11.2, 11.2.1), on Azure Pipelines.

15.10.1.1

Changes:

- **New:** Add direct [Meson Build](https://mesonbuild.com/) (https://mesonbuild.com/) use support. — *Rémi Gastaldi*
- **New:** Define argument value `choices()` as either a fixed set or checked by a custom lambda function. — *René Ferdinand Rivera Morell*
- Fix being able to parse straight from `args` outside of `cli`. Which resulted in misleading parsing behavior. *René Ferdinand Rivera Morell*
- Fix use of `cardinality()` to correctly constrain bounded cardinality ranges like `[0,n]`, `[n,m]`, `[n,n]`, and `[0,0]` (unbounded). *René Ferdinand Rivera Morell*
- Fix use of `required()` arguments such that the parse errors if they are absent. — *girstsf*
- Remove uses of `assert` macro to avoid internal aborts and make it easier to use as a library. — *René Ferdinand Rivera Morell*
- Many documentation improvements. — *René Ferdinand Rivera Morell*
- Tested with Visual Studio 2015, VS 2017, VS 2019, MinGW-64 (gcc 8.1), Linux (clang 3.5, 3.6, 3.7, 3.8, 3.9, 4, 5, 6, 7, 8, 9; gcc 4.8, 4.9, 5, 6, 7, 8, 9), Xcode (9.0, 9.0.1, 9.1, 9.2, 9.3, 9.3.1, 9.4, 9.4.1, 10.0, 10.1, 10.2, 10.2.1, 10.3, 11.0), on Azure Pipelines.

15.11. 1.0

This is the initial base version based on Clara library. Changes from that library:

- Documentation.
- Zero dependencies, even internally, by removing TextFlow and Catch bundled libraries.
- Conform to Pitchfork Layout R1.
- Tested with Visual Studio 2015, VS 2017, MinGW (gcc 5.3), MinGW-64 (gcc 6.3, 7.3, 8.1), Linux (clang 3.5, 3.8, 3.9, 4, 5; gcc 4.8, 4.9, 5, 6, 7, 8), Xcode (8.3, 9, 9.1, 10.1).
- Tested with C++ 11, 14, 17, and 2a.
- **New:** customization of option prefix and separators.
- **New:** support for repeated arguments, from one to many.